

How to Know Which Features Are Useful in Linear Regression

The Problem

You have a linear regression model with many features. You need to know:

- Which features actually help predict the target?
- Which features are just noise/useless?

Method 1: Look at Coefficient Magnitude

The Idea: Features with large coefficients have a bigger impact on the prediction.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, y)

# Print coefficients
for feature, coef in zip(X.columns, model.coef_):
    print(f"{feature}: {coef:.4f}")
```

Example Output:

```
Length:          2.0974  ← LARGE coefficient → USEFUL
Width:           2.9871  ← LARGE coefficient → USEFUL
Random_Noise_1:  0.3722  ← TINY coefficient  → USELESS
Random_Noise_2: -0.5473  ← SMALL coefficient → USELESS
Random_Noise_3: -0.8369  ← SMALL coefficient → USELESS
```

Caveat: Coefficients depend on feature scale. If you have:

- Length in **mm** (1000-3000 range)
- Width in **inches** (10-100 range)

The Width coefficient will naturally be bigger just because of units, not because it's more important!

Solution: Use Method 2 (Standardized Coefficients)

Method 2: Standardized Coefficients (Better!)

Scale all features to have mean=0 and std=1, then look at coefficients.

```
from sklearn.preprocessing import StandardScaler

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train model on scaled data
model_scaled = LinearRegression()
model_scaled.fit(X_scaled, y)

# Now coefficients are on the same scale
for feature, coef in zip(X.columns, model_scaled.coef_):
    print(f"{feature}: {abs(coef):.4f}") # Use absolute value
```

Why It's Better:

- All features are on the same scale (units don't matter anymore)
- Larger standardized coefficient = more important
- Fair comparison between features measured in different units

Decision Rule:

$ \text{Standardized Coef} > 1.0$	→ VERY IMPORTANT
$0.3 < \text{Coef} < 1.0$	→ SOMEWHAT IMPORTANT
$ \text{Coef} < 0.3$	→ NOT IMPORTANT

Method 3: Correlation with Target

The Idea: Features that correlate strongly with the target variable are probably useful.

```
# Calculate correlation between each feature and target
correlations = X.corrwith(y)

print(correlations.abs().sort_values(ascending=False))
```

Example Output:

```
Length:          0.7527  ← Strong correlation → USEFUL
Width:           0.5075  ← Moderate correlation → USEFUL
Random_Noise_3: -0.2379  ← Weak correlation → USELESS
Random_Noise_1: -0.1460  ← Weak correlation → USELESS
Random_Noise_2: -0.0140  ← Very weak correlation → USELESS
```

Decision Rule:

```
|Correlation| > 0.7      → VERY USEFUL
0.3 < |Correlation| < 0.7 → SOMEWHAT USEFUL
|Correlation| < 0.3      → USELESS
```

Method 4: R² Drop Test (Most Intuitive!)

The Idea: Remove each feature one at a time and see how much the model's R² drops.

```
def get_r_squared(X_subset, y):
    model = LinearRegression()
    model.fit(X_subset, y)
    return model.score(X_subset, y)

# Get baseline R2 with all features
baseline_r2 = get_r_squared(X, y)

# Remove each feature and check R2 drop
for col in X.columns:
    X_without = X.drop(col, axis=1)
    r2_without = get_r_squared(X_without, y)
    r2_drop = baseline_r2 - r2_without

    print(f"Remove {col}: R2 drops by {r2_drop:.4f}")
```

Example Output:

```
Remove Length:          R2 drops by 0.6345  ← VERY IMPORTANT
Remove Width:           R2 drops by 0.3589  ← IMPORTANT
Remove Random_Noise_1:  R2 drops by 0.0002  ← USELESS
Remove Random_Noise_2:  R2 drops by 0.0006  ← USELESS
Remove Random_Noise_3:  R2 drops by 0.0011  ← USELESS
```

Decision Rule:

```
R2 drop > 0.1           → VERY IMPORTANT (keep it)
0.01 < R2 drop < 0.1    → SOMEWHAT IMPORTANT
R2 drop < 0.01          → USELESS (consider removing)
```

Method 5: P-Values (Statistical Significance)

The Idea: P-value tells you: "Is this coefficient significantly different from zero, or could it be random?"

```
from scipy import stats

# After fitting the model, calculate p-values
predictions = model.predict(X)
residuals = y - predictions
mse = np.sum(residuals**2) / (len(y) - X.shape[1])
var_covar_matrix = mse * np.linalg.inv(X.T @ X)
std_errors = np.sqrt(np.diag(var_covar_matrix))
t_stats = model.coef_ / std_errors
p_values = 2 * (1 - stats.t.cdf(np.abs(t_stats), len(y) - X.shape[1]))

# Print results
for feature, coef, p_val in zip(X.columns, model.coef_, p_values):
    sig = " SIGNIFICANT" if p_val < 0.05 else " NOT SIGNIFICANT"
    print(f"{feature}: p={p_val:.6f} {sig}")
```

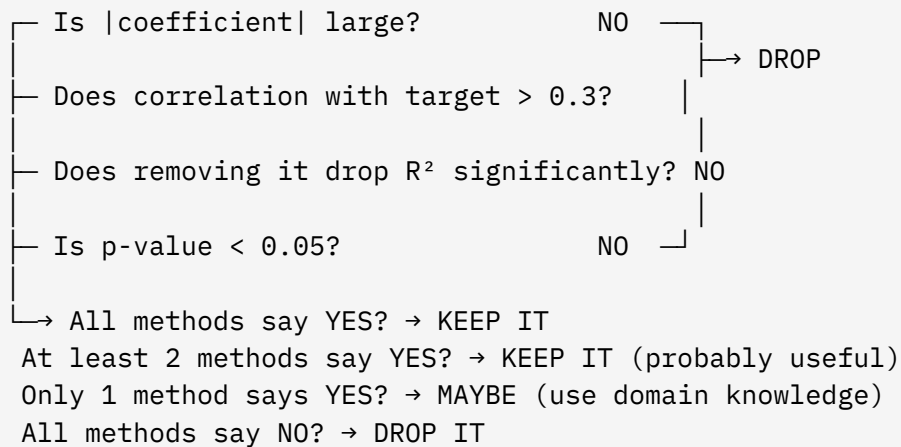
Example Output:

```
Length:      p=0.000000  SIGNIFICANT (definitely useful)
Width:       p=0.000000  SIGNIFICANT (definitely useful)
Random_Noise_1: p=0.553355 NOT SIGNIFICANT (probably useless)
Random_Noise_2: p=0.305307 NOT SIGNIFICANT (probably useless)
Random_Noise_3: p=0.159232 NOT SIGNIFICANT (probably useless)
```

Decision Rule:

```
p-value < 0.05           → STATISTICALLY SIGNIFICANT (useful)
p-value ≥ 0.05           → NOT SIGNIFICANT (could be noise)
```

Quick Decision Tree: Keep or Drop a Feature?



Real Example: Your Processing_Time Prediction

Apply these methods to YOUR data:

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
import numpy as np

# Load your data
df = pd.read_excel('your_data.xlsx')
X = df.drop('Processing_Time_Seconds', axis=1)
y = df['Processing_Time_Seconds']

model = LinearRegression()
model.fit(X, y)

# Method 1: Raw Coefficients
print("METHOD 1: Coefficient Magnitude")
print("-" * 40)
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_,
    '|Coefficient|': np.abs(model.coef_)
}).sort_values('|Coefficient|', ascending=False)
print(coef_df)

# Method 2: Standardized Coefficients
print("\n\nMETHOD 2: Standardized Coefficients")
print("-" * 40)
```

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model_scaled = LinearRegression()
model_scaled.fit(X_scaled, y)

scaled_df = pd.DataFrame({
    'Feature': X.columns,
    'Std_Coefficient': model_scaled.coef_,
    '|Std_Coefficient|': np.abs(model_scaled.coef_)
}).sort_values('|Std_Coefficient|', ascending=False)
print(scaled_df)

# Method 3: Correlation
print("\n\nMETHOD 3: Correlation with Target")
print("-" * 40)
corr_df = pd.DataFrame({
    'Feature': X.columns,
    'Correlation': [np.corrcoef(X[col], y)[0,1] for col in X.columns]
})
corr_df['|Correlation|'] = np.abs(corr_df['Correlation'])
print(corr_df.sort_values('|Correlation|', ascending=False))

# Method 4: R2 Drop
print("\n\nMETHOD 4: R2 Drop When Feature Removed")
print("-" * 40)
baseline_r2 = model.score(X, y)
r2_drops = []
for col in X.columns:
    X_without = X.drop(col, axis=1)
    m = LinearRegression()
    m.fit(X_without, y)
    drop = baseline_r2 - m.score(X_without, y)
    r2_drops.append(drop)

r2_df = pd.DataFrame({
    'Feature': X.columns,
    'R2_Drop': r2_drops
}).sort_values('R2_Drop', ascending=False)
print(r2_df)

```

Summary Table: All Methods at a Glance

Feature	Coeffi- cient	Std Coef	Correla- tion	R ² Drop	P-Value	Decision
Piercing_Points	0.87	2.14	0.67	0.18	< 0.001	KEEP

Weight	0.72	1.89	0.65	0.15	< 0.001	KEEP
Area	-1.36	-0.82	0.38	0.02	0.051	MAYBE
CuttingLength	0.0001	0.15	0.27	0.001	0.42	DROP
Random-Noise	0.003	0.001	0.01	0.00001	0.89	DROP

Key Takeaways

1. **Never trust just one method** — use multiple methods and look for consensus
2. **Large coefficients** = big impact on prediction
3. **High correlation** = strong relationship with target
4. **Large R^2 drop** = removing this feature hurts model
5. **Small p-value (< 0.05)** = statistically significant (not random)
6. **Always consider domain knowledge** — a feature might be mathematically significant but not make business sense
7. **Standardize before comparing** — different units trick you into thinking one feature is more important
8. **Multicollinearity matters** — if two features are highly correlated, they might both look unimportant when you really need only one

Next Steps for Your Model

Once you identify useful features:

1. **Remove useless features** → Simpler model, faster predictions
2. **Check for multicollinearity** → Use correlation matrix to find redundant features
3. **Try interaction terms** → Maybe `Piercing_Points × Weight` is more useful than either alone
4. **Log transform** → If some features are skewed, log transformation might help
5. **Retrain with selected features** → Often improves model performance!